From MQTT to Dashboard

Real-Time IIoT Pipelines with Tiger Cloud

Tiger Data Workshop · July 22, 2026 · Matty Stratton & Doug Pagnutti

Industrial Equipment Never Stops Talking

Temperatures, pressures, flow rates, equipment states: a constant stream of telemetry

Systems built to capture it must keep up without dropping a point

Bridge the gap: real-time visibility into your data

By the end of this workshop you will:

- ✓ Subscribe to MQTT sensors
- ✓ Ingest into TimescaleDB
- ✓ Query with time_bucket()
- ✓ Visualize in Grafana
- ✓ Know how to scale it

Sign up for Tiger Cloud and Github

If you haven't already done so

Tiger Cloud

https://tsdb.co/sensor-workshop-signup

Github

https://github.com/signup

All in the docs.

Quick Start: GitHub Codespaces

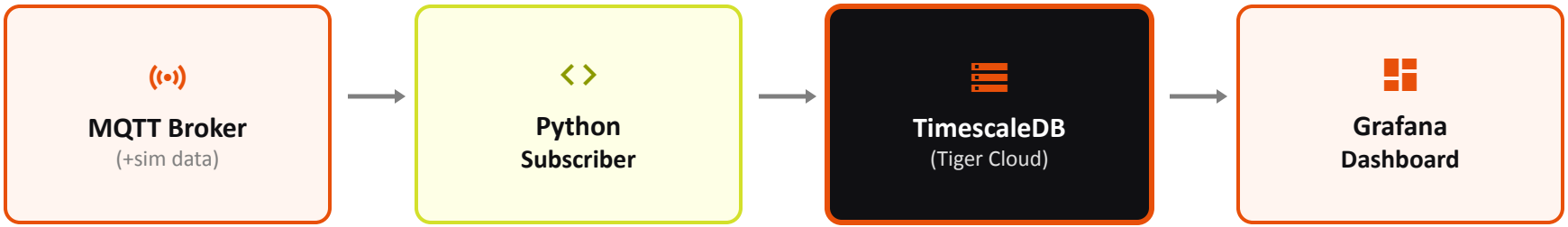
Everything pre-installed. No setup hassle.

https://github.com/timescale/mqtt-grafana-workshop

Includes:

- ✓ Python + dependencies (paho-mqtt, psycopg2)
- ✓ Mosquitto MQTT client tools
- ✓ PostgreSQL client (psql)
- ✓ Grafana running on port 3000

End-to-End Architecture



Real Examples:

- Edge Manufacturing
- Distributed Energy Resources (Solar, Batteries, etc…)
- Mobile Robotics
- Oil and Gas Remote Monitoring
- Facility Management
- …

MQTT: The Protocol

Lightweight. Designed for IIoT and constrained devices.

1. **Publish-Subscribe**
   Publishers send to topics. Subscribers listen.
2. **Topics (hierarchical)**
   e.g., factory/floor1/machine3/temperature
3. **QoS Levels**
   Guarantee message delivery (0, 1, or 2)
4. **Small Overhead**
   Works on slow networks, mobile, edge devices
5. **Broker Pattern**
   Central broker routes all messages

Why MQTT?

- Proven standard in IIoT
- Works over unreliable networks
- Low bandwidth—critical for high-volume sensors
- Decouples producers from consumers
- Scales to millions of devices

MQTT Client Alternatives

Different tools, same MQTT protocol

TimescaleDB & Tiger Cloud

Time-Series Database Optimized for Speed

TimescaleDB extends PostgreSQL with time-series superpowers:

- Hypertables: auto-partitioned on time
- Ingest 1M rows/sec+ per node
- Compression: 90% smaller for old data
- time_bucket() for aggregates
- Continuous aggregates for real-time rollups
- last() for latest-value queries

Tiger Cloud: Managed TimescaleDB

✓ Hosted on AWS / Azure / GCP

✓ Automated backups & replication

✓ Point-in-time recovery

✓ High availability

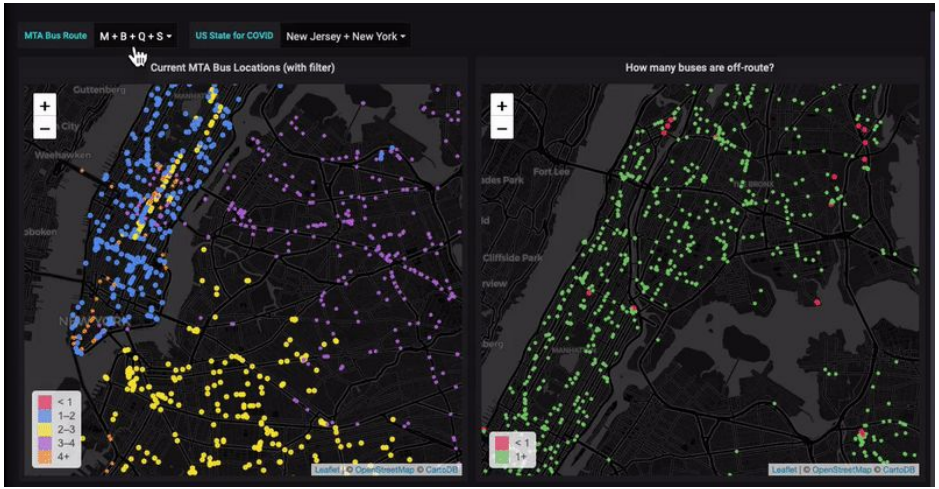
✓ Automatic scaling

✓ Monitoring & alerting

✓ No ops overhead

Grafana

Real-Time Operational Dashboards



https://www.tigerdata.com/blog/grafana-variables-101

Tiger Data

The Workshop

1. Codespace + Credentials

1. **Python 3.8+**
   With paho-mqtt, psycopg2, python-dotenv
2. **Mosquitto**
   mosquitto_sub for testing mqtt
3. **PostgreSQL Client**
   psql for database interaction
4. **Grafana**
   Grafana at localhost:3000

Setup Check

```
Checking Python + dependencies (paho-mqtt, psycopg2,
python-dotenv)...
Checking Mosquitto MQTT client...
Checking PostgreSQL client (psql)...
Checking Grafana (port 3000)...
Success
```

Credentials Configuration

Copy your access credentials into a `.env` file in the root directory before starting.

2. Test MQTT

Connect to the MQTT Broker

```
mosquitto_sub -h 54.160.239.103 -p 1883 -t "UNS/manufacturing/#" -v
```

Output (add `| tee output.txt` to save it to a file) :

UNS/manufacturing/plant1/area1/machine1/humidity {"timestamp": "2026-07-21T19:15:43.480366Z", "tag": "humidity", "value": 48.89, "unit": "%", "description": "Ambient humidity"}

This proves MQTT is working, and shows the message format.

3a. Load .env and Test Tiger Cloud

Step 1: Load credentials (`set -a` exports variables into the environment so other apps can use them)

```
set -a; source .env; set +a
```

Step 2: Connect to Tiger Cloud

```
psql -c '\conninfo'
```

3a. Load .env and test

Two tables: Metadata + Readings

```
CREATE TABLE IF NOT EXISTS tag_meta (
    tag_id VARCHAR(255) PRIMARY KEY,
    tag_name VARCHAR(255) NOT NULL,
    unit VARCHAR(100),
    description TEXT,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE IF NOT EXISTS tag_history (
    time TIMESTAMPTZ NOT NULL,
    tag_id VARCHAR(255) NOT NULL,
    value DOUBLE PRECISION NOT NULL,
    FOREIGN KEY (tag_id) REFERENCES tag_meta(tag_id)
) WITH (
    tsdb.hypertable,
    tsdb.partition_column = 'time'
);
```

Separation of concerns: Metadata rarely changes. Readings flow in constantly.

3b. Schema for Time-Series Data

Two tables: Metadata + Readings

```sql
CREATE TABLE IF NOT EXISTS tag_meta (
    tag_id VARCHAR(255) PRIMARY KEY,
    tag_name VARCHAR(255) NOT NULL,
    unit VARCHAR(100),
    description TEXT,
    created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE TABLE IF NOT EXISTS tag_history (
    time TIMESTAMPTZ NOT NULL,
    tag_id VARCHAR(255) NOT NULL,
    value DOUBLE PRECISION NOT NULL,
    FOREIGN KEY (tag_id) REFERENCES tag_meta(tag_id)
) WITH (
    tsdb.hypertable,
    tsdb.partition_column = 'time'
);
```

Separation of concerns: Metadata rarely changes. Readings flow in constantly.

4. Run Python App

Run the application

```
python3 mqtt_to_timescaledb.py
```

5: Query Sensor Data

Query 1: Latest 10 readings

```
SELECT time, value, tag_id
FROM tag_history
ORDER BY time DESC
LIMIT 10;
```

Query 2: 1-minute averages of a specific tag with time_bucket()

```
SELECT time_bucket('1 minute', h.time) AS minute,
       tag_id,
       avg(value) FROM tag_history
WHERE tag_id = 'plant1/area1/machine1/bearing_temperature'
GROUP BY minute
ORDER BY minute DESC;
```

6: Visualize in Grafana

Step 1: Open Grafana

```
http://localhost:3000
Login: admin / admin
```

Step 2: Add Data Source

Connections → Data Sources → PostgreSQL
Fill in your Tiger Cloud credentials

Step 3: Import Dashboard

Dashboards → Import
Choose: grafana/sensor_readings_dashboard.json

Step 4: Watch it Update

Dashboard refreshes every 10 seconds
See live temperature, pressure curves
One line per tag_id

Key: Grafana queries TimescaleDB with time_bucket() aggregates, so the dashboard stays fast even as you ingest millions of rows.

Scaling & Production Patterns

As your data grows:

Compression

TimescaleDB compresses old data by default—older chunks (24h default) get ~90% smaller

Aggregates

Continuous aggregates pre-compute 1h and 1d rollups. Queries serve the rollup, not raw data

Retention

Archive data older than 1 year. Keep hot data on fast storage

Resources for deeper dives:

TimescaleDB Docs: https://docs.timescale.com/ | Tiger Cloud: https://tigerdata.com/cloud

Your Turn: Apply This Architecture

Take this home and adapt it to your use case:

1. Map to your own MQTT Broker
2. Modify the schema to match your metadata
3. Improve the Python app (batching, quarantines, scaling). Try out other MQTT Clients.
4. Modify and build better dashboards
5. Set up alerting on thresholds
6. Try out Tiered Storage (S3) for greater cost control

Join the Community

https://tsdb.co/exrnuodf

All in the docs.

Questions?

Resources:

GitHub: github.com/timescale | Docs: docs.timescale.com | Community: slack.timescale.com